

Phoenix: Intelligent Personal Financial Analytics Platform

Team 23

1 Description of the Use Case

The primary stakeholders affected by this problem include individuals seeking better financial awareness, financial advisors assisting with financial planning, and banking institutions offering financial analytics services. With the growth of digital payments and online banking, individuals generate large volumes of financial transaction data. However, these records are often fragmented across multiple sources, making it difficult to understand spending behavior or assess overall financial health.

This project addresses this challenge by aggregating transaction data and analyzing it to identify spending patterns, compute financial health indicators, and detect unusual activity. The system presents these insights through a dashboard that visualizes financial trends and provides budgeting recommendations, enabling users to make informed financial decisions and identify potential risks early.

2 Key Functionalities

2.1 Functional Requirements

- **Expense Categorization** Automatically classify transactions into spending categories such as groceries, transportation, utilities, and entertainment.
- **Financial Health Score** Compute a financial health score using metrics such as savings rate, spending trends, and debt levels.
- **Budgeting Assistant** Generate budget recommendations based on historical spending and financial goals.
- **Fraud Detection Alerts** Detect anomalous transactions and notify users of potential fraud or unusual activity.
- **Financial Insights Dashboard** Provide interactive visualizations showing spending patterns, category distributions, and financial health indicators.

2.2 Non-Functional Requirements

- **Security:** Financial data must be encrypted and protected using secure authentication mechanisms.

- **Scalability:** The system must handle increasing transaction volumes and concurrent users.
- **Performance:** Analytics queries and dashboards should respond within seconds.
- **Reliability:** The system must maintain consistent operation despite component failures.
- **Auditability:** All financial data processing operations must be logged for traceability.
- **Maintainability:** The system should support easy updates, bug fixes, and feature modifications.
- **Usability:** The platform should provide a simple and intuitive user interface.

2.3 Technical Implementation

The platform will follow a **microservices-based architecture** with a modular analytics pipeline. The system is organized into three primary layers: a **Frontend Layer**, which provides a React-based dashboard for financial visualization and user interaction; a **Service Layer**, which includes an API gateway and back-end services implemented using **Python (FastAPI)** or **Node.js** to handle transaction ingestion, analytics processing, and recommendation generation; and a **Data Layer**, which manages persistent storage of transaction data, financial metrics, and event logs using a relational or distributed database. Within this architecture, transaction events are processed through a financial analytics engine that computes spending metrics and generates insights that are delivered to the dashboard interface.

3 Design Patterns

The system design applies several well-known GoF design patterns to ensure flexibility, maintainability, and scalability.

1. **Factory Method Pattern** Used to create different types of financial analytics services such as expense categorization, fraud detection, and financial scoring modules without tightly coupling the system to specific implementations.
2. **Adapter Pattern** Allows the system to integrate

with different external banking APIs and payment platforms by adapting various data formats into a unified transaction model.

3. **Facade Pattern** Provides a simplified interface to complex backend services such as analytics processing, transaction ingestion, and recommendation engines.
4. **Observer Pattern** Used for event notifications. When a transaction is processed, observers such as fraud detection services, budgeting modules, and alert systems are notified automatically.
5. **Strategy Pattern** Enables different financial recommendation algorithms (rule-based, statistical models, or machine learning models) to be selected dynamically based on user profiles and financial behavior.

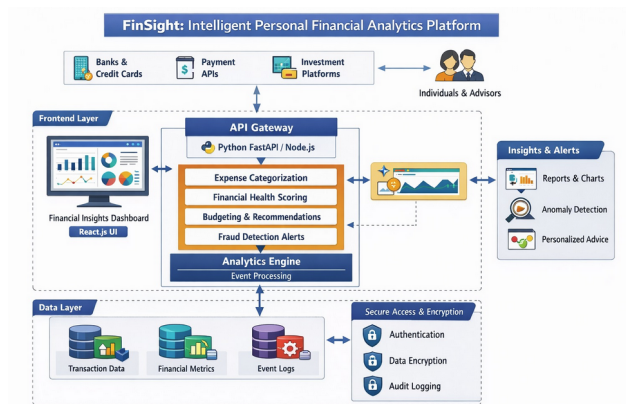


Figure 1: Overall system architecture

4 Architectural Tactics

• Layered Architecture

- Separates presentation, service, and data layers.
- Improves maintainability by isolating responsibilities.
- Enables independent evolution of frontend and backend components.

• Event-Driven Processing

- Processes transaction events through asynchronous analytics pipelines.
- Enables scalable handling of financial data streams.
- Supports near real-time financial insights.

• Security and Data Protection

- Encrypts financial data in transit and at rest.
- Implements secure authentication and identity services.

- Ensures privacy and compliance with financial data standards.

• Fault Tolerance

- Isolates components to prevent cascading failures.
- Uses logging and monitoring for rapid issue detection.
- Employs redundant services to maintain system availability.
- Enforces authentication, authorization, and data anonymization.

• Performance (Caching Tactic)

- Uses an in-memory store (e.g., Redis) to cache financial scores and category distributions.
- Reduces dashboard latency by avoiding repeated heavy database queries.
- Applies a cache-aside strategy to maintain consistency with the primary database.

5 Expected Time to Build a Prototype

The prototype will focus on transaction ingestion and a financial insights dashboard.

Week 1: Research and Planning Problem analysis, financial data modeling, and system architecture design.

Week 2: System Design Design APIs, database schema, security model, and dashboard UI.

Week 3: Development Implement transaction ingestion service, analytics engine, and dashboard components.

Week 4: Integration and Testing Integrate services, test analytics pipelines, optimize performance, and prepare documentation.

Total Estimated Development Time: ~4 weeks.

6 Domain

Domain: Financial Technology (FinTech)

The project lies within the FinTech domain and focuses on improving personal financial management through intelligent analytics systems. By combining financial data aggregation, event-driven processing, and analytics-based recommendations, the platform enables users to better understand spending behavior and maintain long-term financial health.